

UNIVERSIDADE ESTADUAL DE PONTA GROSSA – UEPG
SETOR DE ENGENHARIAS, CIÊNCIAS AGRÁRIAS E TECNOLOGIA
DEPARTAMENTO DE INFORMÁTICA - BACHARELADO EM ENGENHARIA DE
COMPUTAÇÃO

JOÃO PEDRO BORSATO DE RAMOS
JOÃO VITOR CARVALHO DE ANDRADE

OASIS:
PROJETO DE SOFTWARE DE UMA APLICAÇÃO WEB PARA GESTÃO
DE RESERVAS E CONTROLE DE ACESSO EM CONDOMÍNIOS RESIDENCIAIS

PONTA GROSSA
2026

JOÃO PEDRO BORSATO DE RAMOS
JOÃO VITOR CARVALHO DE ANDRADE

OASIS:
**PROJETO DE SOFTWARE DE UMA APLICAÇÃO WEB PARA GESTÃO
DE RESERVAS E CONTROLE DE ACESSO EM CONDOMÍNIOS RESIDENCIAIS**

Projeto de Software apresentado ao curso de Engenharia de Computação da Universidade Estadual de Ponta Grossa como requisito parcial na disciplina de Projeto de Software.

Professores: Diolete Marcante Lati Cerutti, Ezequiel Gueiber e Idomar Augusto Cerutti.

SUMÁRIO

1 DIAGNÓSTICO ATUAL	4
2 OBJETIVOS DO PROJETO	4
2.1 Objetivo Geral	4
2.2 Objetivos Específicos	5
3 ESCOPO	5
4 PROPOSTA DE PROJETO	7
4.1 Benefícios	8
4.2 Custos	9
4.3 Riscos	10
4.4 Modelagem do Sistema	11
4.4.1 Diagrama de Casos de Uso	11
4.4.2 Diagrama de Classes	19
4.4.3 Diagrama de Sequência	21
4.5 Modelagem do Banco de Dados	25
4.5.1 Diagrama Entidade-Relacionamento	25
4.5.2 Diagrama Lógico	26
4.5.3 Dicionário de Dados	26
4.5.4 Scripts do Banco de Dados	32
4.5.5 Regras de Negócio e Gatilhos	35
4.6 Protótipo de Interface	38
4.7 Plano de Testes	41
5 CRONOGRAMA	44
6 CONCLUSÃO	45
7 REFERÊNCIAS	46
8 RESPONSABILIDADES	47

1 DIAGNÓSTICO ATUAL

O cliente deste projeto é um condomínio residencial localizado em Ponta Grossa, representado pela síndica responsável pela administração. Atualmente, o condomínio utiliza o aplicativo comercial BRCONDOS para a gestão de reservas (agendamentos) de espaços comuns. Embora a ferramenta disponibilize um conjunto de funcionalidades, a síndica relatou que o aplicativo não contempla o acesso biométrico associado à reserva do espaço (academia, piscina), nem o acesso por reconhecimento facial à garagem e à porta principal de entrada do condomínio.

Outro problema relatado é que o sistema atual restringe cada apartamento a somente 2 logins de acesso, mesmo quando há mais de 2 moradores na unidade. Um terceiro morador não pode ter login para fazer a reserva em seu nome. Desse modo, caso um morador de determinado apartamento efetue uma reserva da academia no período matutino, os demais moradores da mesma unidade ficam impedidos de realizar uma reserva complementar para outro horário do mesmo dia, mesmo que os espaços estejam disponíveis. Essa limitação gera conflitos internos e insatisfação entre os condôminos.

Quanto ao sistema de comunicação e notificações, a transmissão de avisos da administração aos moradores ocorre atualmente de forma informal, por meio de aplicativos de mensagens (WhatsApp) e cartazes físicos afixados em áreas comuns. Não há um canal centralizado, oficial e rastreável para a comunicação condominial, tampouco mecanismos automatizados de notificação para a chegada de encomendas, o que dificulta a gestão de informações e o acompanhamento pelos moradores.

2 OBJETIVOS DO PROJETO

2.1 Objetivo Geral

Desenvolver uma aplicação web denominada OASIS destinada à gestão integrada de reservas de espaços coletivos e ao controle de acesso biométrico em

condomínios residenciais, promovendo maior praticidade para os moradores e maior organização, transparência e monitoramento para a administração condominial.

2.2 Objetivos Específicos

Os objetivos específicos do projeto são:

- Eliminar a restrição de múltiplos logins por apartamento, possibilitando o controle individualizado de reservas e acessos por morador;
- Centralizar e formalizar a comunicação entre a administração condominial e os moradores, substituindo o uso de canais informais por um meio oficial e rastreável;
- Integrar a gestão de reservas ao sistema de controle de acesso biométrico, garantindo que apenas moradores autorizados acessem os espaços nos horários reservados;
- Reduzir conflitos no uso dos espaços coletivos por meio de regras configuráveis de antecedência, limite de reservas por unidade e bloqueio programado para manutenção;
- Fornecer à administração condominial dados consolidados sobre o uso dos espaços, fluxo de moradores e movimentação de encomendas, apoiando decisões de gestão;
- Automatizar o processo de notificação aos moradores sobre reservas confirmadas, avisos da administração e chegada de encomendas.

3 ESCOPO

O OASIS abrangerá as seguintes funcionalidades dentro de seu escopo de desenvolvimento:

- Cadastro e autenticação de três perfis distintos de usuários: morador, administração (síndica) e porteiro;
- Gerenciamento de espaços coletivos, incluindo cadastro, edição, listagem de utensílios disponíveis e definição de janelas de bloqueio programado para manutenção ou limpeza;

- Reserva de espaços coletivos por morador (não por apartamento), com visualização de disponibilidade em calendário, controle de conflitos de horário, regras de antecedência mínima e limite de reservas simultâneas por unidade;
- Cancelamento de reserva com prazo configurável definido pela administração;
- Agendamento de horários para mudanças, com definição de janelas de tempo e limitação de vagas simultâneas;
- Mural de avisos: publicação, edição e remoção de comunicados pela administração, com visualização pelos moradores;
- Controle de recebimento e entrega de encomendas pela portaria, incluindo registro de data de recebimento, data de retirada e notificação automática ao morador destinatário;
- Integração com a plataforma Intelbras para liberação de acesso por biometria e reconhecimento facial;
- Sistema de notificações *push* para alertar moradores sobre reservas, avisos e encomendas;
- Painel administrativo com relatórios de métricas de uso dos espaços, encomendas e fluxo de moradores.

Estão fora do escopo do presente projeto:

- Aplicativo mobile nativo (Android ou iOS) — a aplicação será exclusivamente web, com layout responsivo para acesso via navegadores em dispositivos móveis;
- Integração com sistemas de cobrança ou pagamento de taxas condominiais;
- Gestão financeira do condomínio (controle de receitas, despesas, inadimplência);
- Controle de acesso físico (portaria eletrônica, câmeras de segurança);
- Módulo de votação em assembleia;
- Chat em tempo real entre moradores;
- Integração com o sistema BRCONDOS.

As funcionalidades fora do escopo não serão atendidas neste projeto em razão das restrições de tempo e de recursos da equipe de desenvolvimento, bem como por

não fazerem parte das demandas prioritárias identificadas junto à cliente. Tais funcionalidades poderão ser incorporadas em versões futuras do sistema.

4 PROPOSTA DE PROJETO

O OASIS será desenvolvido exclusivamente como uma aplicação web responsiva, acessível através de navegadores em computadores e dispositivos móveis. A escolha pela plataforma web exclusiva visa reduzir os custos do projeto e simplificar o processo de manutenção e atualização do sistema, sem prejuízo da experiência do usuário em diferentes dispositivos. A seguir, são apresentadas as tecnologias, *frameworks* e ferramentas que serão utilizadas no desenvolvimento, com suas respectivas versões.

Frontend (Interface Web)

- *React* 18.3 (LTS) — biblioteca JavaScript para construção da interface de usuário;
- *TypeScript* 5.5 — linguagem tipada baseada em JavaScript, para maior segurança no desenvolvimento;
- *Vite* 5.4 — ferramenta de *build* e *bundler* moderno para projetos React;
- *Tailwind CSS* 3.4 — *framework* de estilização baseado em classes utilitárias.

Backend (API e Lógica de Negócio)

- *Node.js* 22.11 (LTS) — ambiente de execução JavaScript para o servidor;
- *NestJS* 10.4 — *framework* progressivo para construção de aplicações backend escaláveis em Node.js;
- *Prisma* 5.20 — ORM (*Object-Relational Mapping*) moderno para acesso ao banco de dados.

Banco de Dados

- *PostgreSQL* 16 — banco de dados relacional para persistência de dados estruturados;

- Hospedagem em Supabase ou Railway (planos gratuitos para projetos acadêmicos).

Autenticação e Notificações

- *Firebase Authentication* — serviço gerenciado para autenticação de usuários por e-mail e senha;
- *Firebase Cloud Messaging (FCM)* — serviço de envio de notificações *push* em tempo real.

Integração com Controle de Acesso Físico

- API da plataforma Intelbras — responsável pelo acionamento dos leitores biométricos e dispositivos de reconhecimento facial instalados no condomínio. A integração permitirá que o OASIS envie comandos para liberação de acesso conforme as reservas confirmadas no sistema.

Infraestrutura e Hospedagem

- *Vercel* — hospedagem do *frontend* web (plano gratuito);
- *Railway* ou *Render* — hospedagem do *backend* (planos gratuitos);
- *GitHub* — versionamento de código e colaboração.

Ferramentas Auxiliares

- *Visual Studio Code* — editor de código (gratuito);
- *Figma* — prototipação de interface (plano gratuito);
- *Postman 11* — testes manuais de endpoints da API;
- *draw.io*, *BRModelo Web* e *Mermaid* — modelagem de diagramas UML, DER e diagrama lógico.

4.1 BENEFÍCIOS

A implantação do OASIS proporcionará os seguintes benefícios ao condomínio:

- Eliminação das restrições de reserva por apartamento, permitindo que todos os moradores de uma unidade agendem os espaços coletivos de forma independente, aumentando a equidade e reduzindo conflitos internos;
- Centralização e formalização da comunicação condominial em um canal oficial, organizado e com histórico acessível;
- Redução do tempo despendido pela administração no gerenciamento manual de reservas e na resolução de conflitos de agendamento;
- Maior transparência na gestão das reservas, permitindo que os moradores acompanhem o status de suas solicitações sem necessidade de contato direto com a administração;
- Acompanhamento da chegada e da retirada de encomendas, com notificação automática ao morador destinatário;
- Interface simplificada e intuitiva, aumentando a taxa de adoção do sistema por moradores de diferentes perfis e faixas etárias;
- Disponibilidade multi-dispositivo, garantindo acesso ao sistema independentemente do dispositivo (computador, tablet ou smartphone) utilizado pelo morador.

4.2 CUSTOS

Os custos do projeto foram estruturados em dois grupos: esforço intelectual (mão de obra) e infraestrutura/serviços de terceiros. Embora o projeto seja desenvolvido em contexto acadêmico, sem remuneração direta aos desenvolvedores, os valores foram calculados com base em referências de mercado a fim de representar o custo real equivalente do desenvolvimento, conforme demonstrado na Tabela 1.

A estimativa de esforço foi realizada com base na dedicação parcial de dois desenvolvedores ao longo de aproximadamente 9 meses (março a novembro de 2026), considerando uma carga média de 80 horas mensais por desenvolvedor, compatível com a carga acadêmica da disciplina. O valor de referência por hora foi obtido na plataforma Glassdoor (2026), que aponta a média salarial de um desenvolvedor pleno no Brasil em R\$ 40,00/hora. Os valores de infraestrutura

referem-se a serviços de hospedagem e ferramentas que oferecem planos gratuitos compatíveis com o porte do projeto acadêmico.

Tabela 1 – Custos estimados do projeto

Categoria	Item / Descrição	Valor (R\$)
Esforço Intelectual	Mão de obra — 2 desenvolvedores × 9 meses × 80h = 1.440 horas (R\$ 40,00/h)	R\$ 57.600,00
	Subtotal — Esforço Intelectual	R\$ 57.600,00
Infraestrutura	Hospedagem do backend — Railway ou Render (plano gratuito)	R\$ 0,00
Infraestrutura	Hospedagem do frontend web — Vercel (plano gratuito)	R\$ 0,00
Infraestrutura	Banco de dados — PostgreSQL via Supabase ou Railway (plano gratuito)	R\$ 0,00
Infraestrutura	Firebase — autenticação e notificações push (plano Spark)	R\$ 0,00
Infraestrutura	Ferramentas de desenvolvimento — VS Code, Figma, GitHub (planos gratuitos)	R\$ 0,00
Subtotal Infra	Subtotal — Infraestrutura e Terceiros	R\$ 0,00
TOTAL	CUSTO TOTAL ESTIMADO DO PROJETO	R\$ 57.600,00

Fonte: Os autores. Referência salarial: Glassdoor (2026).

4.3 RISCOS

Conforme metodologia de análise de riscos proposta por Pressman e Maxim (2016), foram identificados os principais riscos do projeto, classificados em termos de probabilidade e impacto, com suas respectivas estratégias de mitigação. A Tabela 2 apresenta a matriz consolidada de riscos identificados (PRESSMAN; MAXIM, 2016).

Tabela 2 – Matriz de riscos do projeto

Risco	Probabilidade	Impacto	Estratégia de Mitigação
Escopo ampliado além da capacidade da equipe	Média	Alto	Definição clara do MVP e revisão periódica das funcionalidades prioritárias a cada banca (PRESSMAN; MAXIM, 2016).
Dificuldade na integração com a API da Intelbras	Média	Alto	Mapear a documentação da API da Intelbras antecipadamente e realizar testes de integração nas fases iniciais (PRESSMAN; MAXIM, 2016).
Dificuldade de adaptação da cliente ao novo sistema	Baixa	Médio	Treinamento presencial e elaboração de manual de uso simplificado (PRESSMAN; MAXIM, 2016).
Indisponibilidade dos serviços de hospedagem gratuitos	Baixa	Médio	Manter alternativas mapeadas: Render, Railway e Supabase (PRESSMAN; MAXIM, 2016).
Conflitos de agenda entre os membros da equipe	Média	Médio	Uso de metodologia ágil com sprints semanais e ferramenta colaborativa de gestão de tarefas (PRESSMAN; MAXIM, 2016).
Mudança nos requisitos por parte da cliente	Alta	Alto	Documentação formal dos requisitos com assinatura da cliente e controle de mudanças registrado (PRESSMAN; MAXIM, 2016).
Desistência de um membro da equipe	Baixa	Alto	Distribuição equilibrada de tarefas e documentação contínua de todo o progresso (PRESSMAN; MAXIM, 2016).
Falhas ou bugs após implementação	Média	Alto	Testes contínuos, revisão de código e período de homologação antes de cada banca (PRESSMAN; MAXIM, 2016).

Fonte: Os autores, com base em Pressman e Maxim (2016).

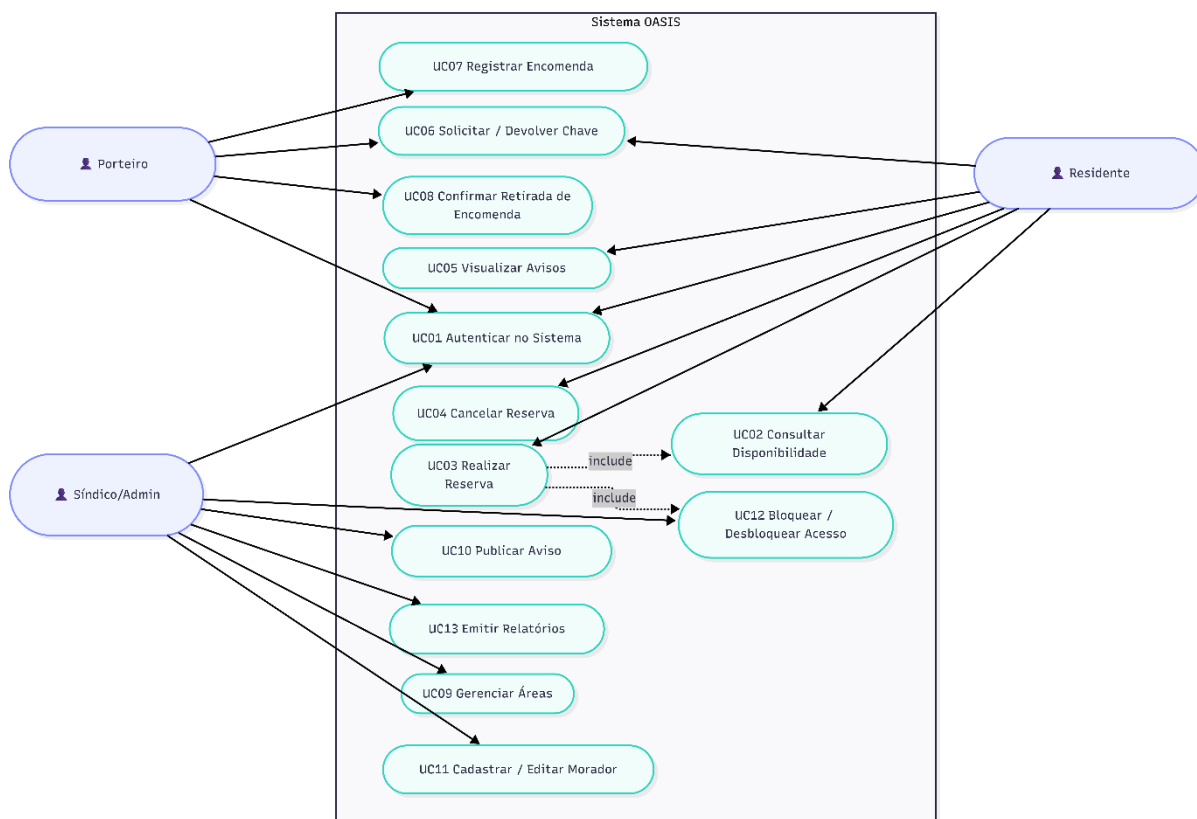
4.4 MODELAGEM DO SISTEMA

A modelagem do OASIS adota a abordagem orientada a objetos com notação UML (Unified Modeling Language). São apresentados a seguir o Diagrama de Casos de Uso com suas descrições detalhadas, o Diagrama de Classes e o Diagrama de Sequência. Os diagramas de banco de dados, o Dicionário de Dados e os scripts de criação são apresentados na seção 4.5.

4.4.1 Diagrama de Casos de Uso

O Diagrama de Casos de Uso, apresentado na Figura 1, representa as interações entre os três perfis de atores humanos — Morador (Residente), Síndico/Administração e Porteiro — e o sistema OASIS. Os casos de uso são detalhados individualmente nos Quadros 1 a 13, conforme a notação proposta por Fowler (2004). Este diagrama foi elaborado com o auxílio da ferramenta Mermaid (2026), que permite a geração de diagramas a partir de descrição textual.

Figura 1 – Diagrama de Casos de Uso do Software OASIS



Fonte: Os autores.

Quadro 1 – Descrição do UC01: Autenticar no Sistema

Identificador	Descrição do Caso de Uso
Identificador	UC01
Nome	Autenticar no Sistema
Ator(es)	Morador (Residente) / Síndico/Administração / Porteiro
Descrição	Permite a qualquer usuário do sistema realizar autenticação por meio de e-mail e senha, via Firebase Authentication, sendo redirecionado ao painel correspondente ao seu perfil.
Pré-condição	Usuário possui cadastro ativo no sistema.
Fluxo Principal	1. Usuário acessa a tela de login. 2. Informa e-mail e senha. 3. Sistema encaminha as credenciais ao Firebase Authentication. 4. Firebase valida as credenciais. 5. Sistema redireciona o usuário ao painel correspondente ao seu perfil.

Fluxo Alternativo	A1 – Credenciais inválidas: sistema exibe mensagem de erro e mantém na tela de login. A2 – Conta inativa: sistema exibe mensagem informando que a conta está desativada.
Pós-condição	Usuário autenticado e redirecionado ao painel correspondente ao seu perfil.

Fonte: Os autores.

Quadro 2 – Descrição do UC02: Consultar Disponibilidade

Identificador	Descrição do Caso de Uso
Identificador	UC02
Nome	Consultar Disponibilidade
Ator(es)	Morador (Residente)
Descrição	Permite ao morador consultar os horários disponíveis de uma área comum em uma data específica, considerando reservas existentes e bloqueios programados.
Pré-condição	Morador autenticado no sistema.
Fluxo Principal	1. Morador acessa o módulo de reservas. 2. Seleciona uma área comum. 3. Seleciona uma data no calendário. 4. Sistema exibe os horários disponíveis e os já ocupados para a data selecionada.
Fluxo Alternativo	A1 – Área comum bloqueada para manutenção: sistema exibe aviso de indisponibilidade com justificativa e período de bloqueio.
Pós-condição	Disponibilidade exibida ao morador com os horários livres e ocupados.

Fonte: Os autores.

Quadro 3 – Descrição do UC03: Realizar Reserva

Identificador	Descrição do Caso de Uso
Identificador	UC03
Nome	Realizar Reserva
Ator(es)	Morador (Residente)
Descrição	Permite ao morador reservar uma área comum em horário disponível, respeitando regras de antecedência e limite de reservas por unidade. Inclui a consulta de disponibilidade (UC02) e o controle de acesso (UC12). A reserva do elevador de serviço para mudanças também é realizada por este caso de uso.

Pré-condição	Morador autenticado; área comum disponível no horário selecionado; morador não excedeu o limite de reservas simultâneas.
Fluxo Principal	1. Morador consulta os horários disponíveis (UC02 «include»). 2. Seleciona área comum, data e horário. 3. Sistema verifica a regra de antecedência mínima. 4. Sistema verifica o limite de reservas simultâneas por unidade. 5. Morador confirma a reserva. 6. Sistema registra a reserva com status 'Ativa'. 7. Sistema aciona o controle de acesso para o período reservado (UC12 «include»).
Fluxo Alternativo	A1 – Limite de reservas excedido: sistema bloqueia e exibe mensagem de aviso. A2 – Horário já reservado: sistema impede a reserva e solicita novo horário.
Pós-condição	Reserva registrada com status 'Ativa'; acesso liberado para o período reservado.

Fonte: Os autores.

Quadro 4 – Descrição do UC04: Cancelar Reserva

Identificador	Descrição do Caso de Uso
Identificador	UC04
Nome	Cancelar Reserva
Ator(es)	Morador (Residente)
Descrição	Permite ao morador cancelar uma reserva ativa dentro do prazo de cancelamento configurado pela administração.
Pré-condição	Morador autenticado; reserva ativa dentro do prazo de cancelamento.
Fluxo Principal	1. Morador acessa a lista de suas reservas. 2. Seleciona a reserva a ser cancelada. 3. Sistema verifica se está dentro do prazo de cancelamento. 4. Morador confirma o cancelamento. 5. Sistema atualiza o status para 'Cancelada' e registra a data de cancelamento. 6. Sistema libera o horário para novas reservas.
Fluxo Alternativo	A1 – Fora do prazo: sistema exibe mensagem informando que o cancelamento não é mais permitido.
Pós-condição	Reserva cancelada; horário liberado para novas reservas.

Fonte: Os autores.

Quadro 5 – Descrição do UC05: Visualizar Avisos

Identificador	Descrição do Caso de Uso
Identificador	UC05
Nome	Visualizar Avisos
Ator(es)	Morador (Residente)
Descrição	Permite ao morador acessar e ler os comunicados publicados pela administração no mural digital.
Pré-condição	Morador autenticado no sistema.
Fluxo Principal	1. Morador acessa o mural de avisos. 2. Sistema exibe os comunicados em ordem cronológica decrescente.
Fluxo Alternativo	A1 – Sem avisos cadastrados: sistema exibe mensagem informando que não há comunicados disponíveis.
Pós-condição	Avisos exibidos ao morador em ordem cronológica.

Fonte: Os autores.

Quadro 6 – Descrição do UC06: Solicitar / Devolver Chave

Identificador	Descrição do Caso de Uso
Identificador	UC06
Nome	Solicitar / Devolver Chave
Ator(es)	Morador (Residente) / Porteiro
Descrição	Permite o registro do empréstimo e da devolução de chaves de áreas comuns. O morador solicita a chave na portaria e o porteiro registra a retirada e, posteriormente, a devolução.
Pré-condição	Morador e porteiro autenticados; chave cadastrada e disponível.
Fluxo Principal	1. Morador solicita a chave de uma área comum na portaria. 2. Porteiro localiza a chave no sistema. 3. Porteiro registra o empréstimo, vinculando a chave ao morador solicitante. 4. Sistema atualiza o status da chave para 'Emprestada' e registra data e hora. 5. Na devolução, o porteiro registra o retorno e o sistema atualiza o status para 'Disponível'.
Fluxo Alternativo	A1 – Chave já emprestada: sistema exibe o morador responsável e impede novo empréstimo.
Pós-condição	Empréstimo ou devolução registrado; status da chave atualizado.

Fonte: Os autores.

Quadro 7 – Descrição do UC07: Registrar Encomenda

Identificador	Descrição do Caso de Uso
Identificador	UC07
Nome	Registrar Encomenda
Ator(es)	Porteiro
Descrição	Permite ao porteiro registrar o recebimento de uma encomenda destinada a um morador, acionando notificação automática.
Pré-condição	Porteiro autenticado; morador destinatário cadastrado no sistema.
Fluxo Principal	1. Porteiro acessa o módulo de encomendas. 2. Busca o morador destinatário pelo nome ou apartamento. 3. Informa a descrição da encomenda. 4. Sistema registra com data/hora de recebimento e status 'Aguardando Retirada'. 5. Sistema envia notificação push ao morador via Firebase FCM.
Fluxo Alternativo	A1 – Morador não encontrado: sistema exibe mensagem de erro.
Pós-condição	Encomenda registrada com status 'Aguardando Retirada'; morador notificado.

Fonte: Os autores.

Quadro 8 – Descrição do UC08: Confirmar Retirada de Encomenda

Identificador	Descrição do Caso de Uso
Identificador	UC08
Nome	Confirmar Retirada de Encomenda
Ator(es)	Porteiro
Descrição	Permite ao porteiro confirmar que o morador retirou a encomenda, atualizando o registro com data e hora da retirada.
Pré-condição	Porteiro autenticado; encomenda com status 'Aguardando Retirada'.
Fluxo Principal	1. Porteiro localiza a encomenda no sistema. 2. Morador se apresenta para retirada. 3. Porteiro confirma a retirada no sistema. 4. Sistema registra a data e hora da retirada e atualiza o status para 'Retirada'.
Fluxo Alternativo	Não há fluxo alternativo identificado.
Pós-condição	Encomenda marcada como retirada com data e hora registradas.

Fonte: Os autores.

Quadro 9 – Descrição do UC09: Gerenciar Áreas Comuns

Identificador	Descrição do Caso de Uso
Identificador	UC09
Nome	Gerenciar Áreas Comuns
Ator(es)	Síndico/Administração
Descrição	Permite à administração cadastrar, editar ou remover áreas comuns, configurar seus horários, utensílios disponíveis e janelas de bloqueio programado para manutenção ou limpeza.
Pré-condição	Administração autenticada no sistema.
Fluxo Principal	1. Administração acessa o módulo de gestão de áreas comuns. 2. Seleciona a operação desejada (cadastrar, editar ou remover). 3. Para cadastro/edição: informa nome, descrição, capacidade, horários, intervalo de reserva e lista de utensílios. 4. Sistema valida e registra as alterações.
Fluxo Alternativo	A1 – Tentativa de remoção com reservas ativas: sistema impede a remoção e exibe aviso.
Pós-condição	Área comum atualizada no sistema conforme a operação realizada.

Fonte: Os autores.

Quadro 10 – Descrição do UC10: Publicar Aviso

Identificador	Descrição do Caso de Uso
Identificador	UC10
Nome	Publicar Aviso
Ator(es)	Síndico/Administração
Descrição	Permite à administração publicar comunicados no mural digital do sistema, com envio automático de notificação push aos moradores.
Pré-condição	Administração autenticada no sistema.
Fluxo Principal	1. Administração acessa o mural de avisos. 2. Cria novo aviso informando título e conteúdo. 3. Publica o aviso. 4. Sistema registra e exibe o aviso no mural. 5. Sistema envia notificação push a todos os moradores via Firebase FCM.
Fluxo Alternativo	Não há fluxo alternativo identificado.
Pós-condição	Aviso publicado no mural e moradores notificados via push.

Fonte: Os autores.

Quadro 11 – Descrição do UC11: Cadastrar / Editar Morador

Identificador	Descrição do Caso de Uso
Identificador	UC11
Nome	Cadastrar / Editar Morador
Ator(es)	Síndico/Administração
Descrição	Permite à administração cadastrar um novo morador no sistema ou editar os dados de um morador existente, fornecendo seus dados pessoais e de unidade.
Pré-condição	Administração autenticada no sistema.
Fluxo Principal	1. Administração acessa o módulo de gestão de moradores. 2. Seleciona cadastrar novo morador ou editar um existente. 3. Preenche nome, apartamento, bloco, e-mail, data de nascimento e telefone. 4. Sistema valida os dados informados. 5. Sistema registra ou atualiza o morador. 6. No cadastro, o sistema envia e-mail com credenciais de acesso ao morador.
Fluxo Alternativo	A1 – E-mail já cadastrado: sistema exibe mensagem de erro e não realiza o cadastro.
Pós-condição	Morador cadastrado ou atualizado, com acesso liberado ao sistema.

Fonte: Os autores.

Quadro 12 – Descrição do UC12: Bloquear / Desbloquear Acesso

Identificador	Descrição do Caso de Uso
Identificador	UC12
Nome	Bloquear / Desbloquear Acesso
Ator(es)	Síndico/Administração
Descrição	Permite à administração bloquear ou desbloquear o acesso de um morador às áreas comuns, refletindo a liberação ou o impedimento nos dispositivos de controle de acesso integrados (Intelbras). É acionado de forma incluída pela reserva (UC03) para liberar o acesso no período reservado.
Pré-condição	Administração autenticada; morador cadastrado no sistema.
Fluxo Principal	1. Administração seleciona o morador no painel de controle de acesso. 2. Define a ação de bloquear ou desbloquear o acesso. 3. Sistema registra a alteração de permissão. 4. Sistema sincroniza a permissão com os dispositivos de controle de acesso integrados.

Fluxo Alternativo	A1 – Falha de comunicação com o dispositivo de acesso: sistema registra log de erro e agenda nova tentativa de sincronização.
Pós-condição	Permissão de acesso do morador atualizada e sincronizada com os dispositivos físicos.

Fonte: Os autores.

Quadro 13 – Descrição do UC13: Emitir Relatórios

Identificador	Descrição do Caso de Uso
Identificador	UC13
Nome	Emitir Relatórios
Ator(es)	Síndico/Administração
Descrição	Permite à administração gerar relatórios gerenciais consolidados sobre o uso das áreas comuns, o histórico de reservas, a movimentação de encomendas e o empréstimo de chaves.
Pré-condição	Administração autenticada no sistema.
Fluxo Principal	1. Administração acessa o módulo de relatórios. 2. Seleciona o tipo de relatório e o período desejado. 3. Sistema consolida os dados registrados no período informado. 4. Sistema exibe o relatório com indicadores e permite a exportação.
Fluxo Alternativo	A1 – Sem dados no período: sistema exibe mensagem informando a ausência de registros.
Pós-condição	Relatório gerado e disponibilizado para visualização e exportação.

Fonte: Os autores.

4.4.2 Diagrama de Classes

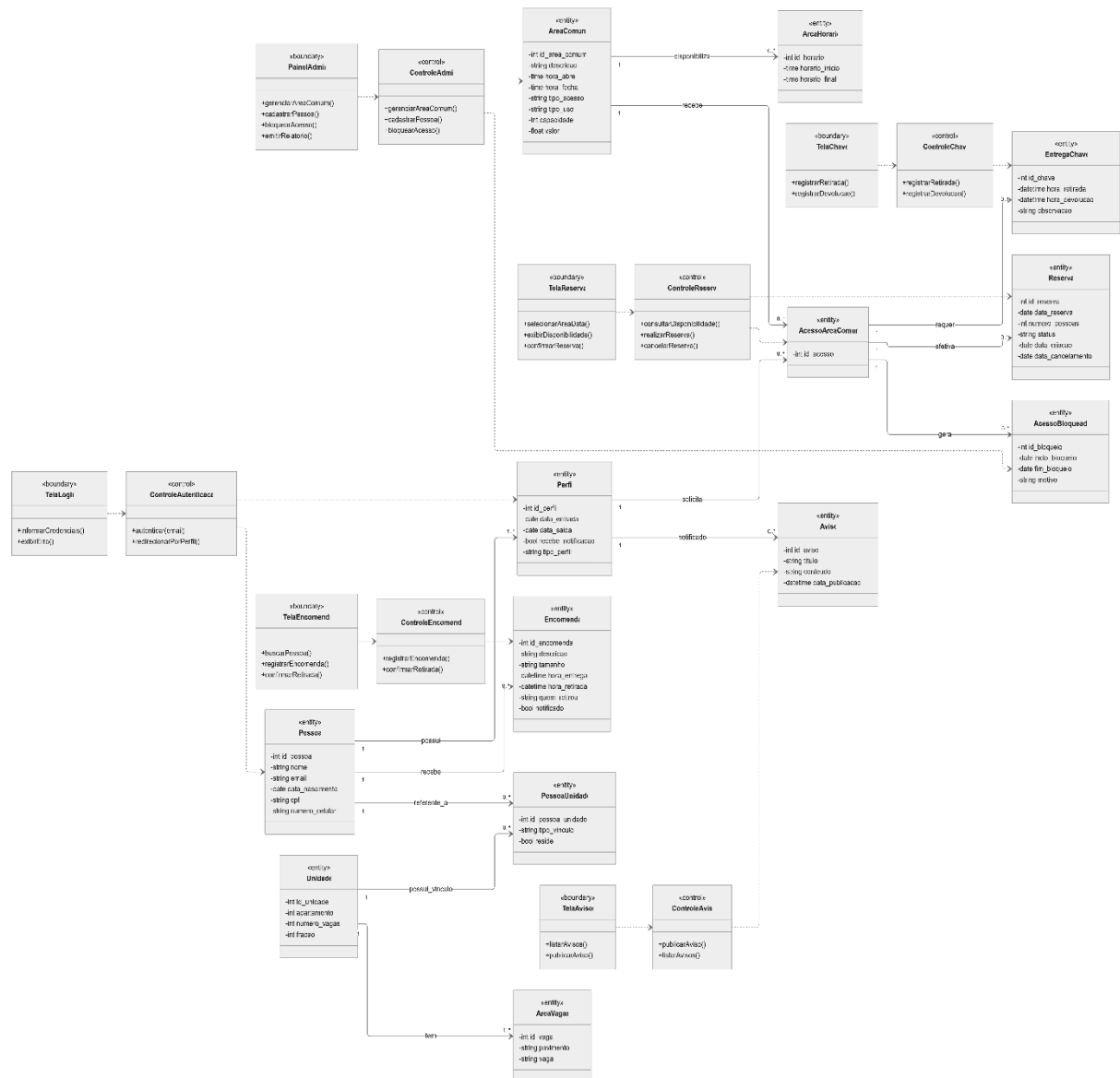
O Diagrama de Classes, apresentado na Figura 2, representa a estrutura estática do sistema OASIS seguindo a notação UML proposta por Fowler (2004). O diagrama foi organizado de acordo com o padrão de análise Boundary-Control-Entity (BCE), que separa as classes em três estereótipos conforme sua responsabilidade, favorecendo a coesão e o baixo acoplamento entre a interface, a lógica de negócio e os dados.

As classes de fronteira («boundary») representam as telas e interfaces por meio das quais os atores interagem com o sistema, sendo elas: TelaLogin, TelaReserva,

TelaChave, TelaEncomenda, TelaAvisos e PaineAdmin. As classes de controle («control») coordenam a execução dos casos de uso, intermediando a comunicação entre as fronteiras e as entidades, sendo elas: ControleAutenticacao, ControleReserva, ControleChave, ControleEncomenda, ControleAviso e ControleAdmin. As classes de entidade («entity») representam os dados persistidos no banco, correspondendo às classes PESSOA, PERFIL, UNIDADE, PESSOA_UNIDADE, AREA_VAGAS, AREA_COMUM, AREA_HORARIO, ACESSO_AREA_COMUM, RESERVA, ENTREGA_CHAVE, ACESSO_BLOQUEADO, ENCOMENDA e AVISO. A classe PESSOA centraliza os dados comuns aos usuários, enquanto a classe PERFIL define o papel de cada pessoa no condomínio (morador, síndico ou porteiro).

O fluxo de dependências segue o sentido fronteira → controle → entidade: cada classe de fronteira aciona a classe de controle correspondente, que por sua vez manipula as classes de entidade. Os relacionamentos de associação entre as entidades preservam as multiplicidades definidas na modelagem do banco de dados (seção 4.5), enquanto a herança e as dependências completam a estrutura conforme a notação UML.

Figura 2 – Diagrama de Classes do Software OASIS

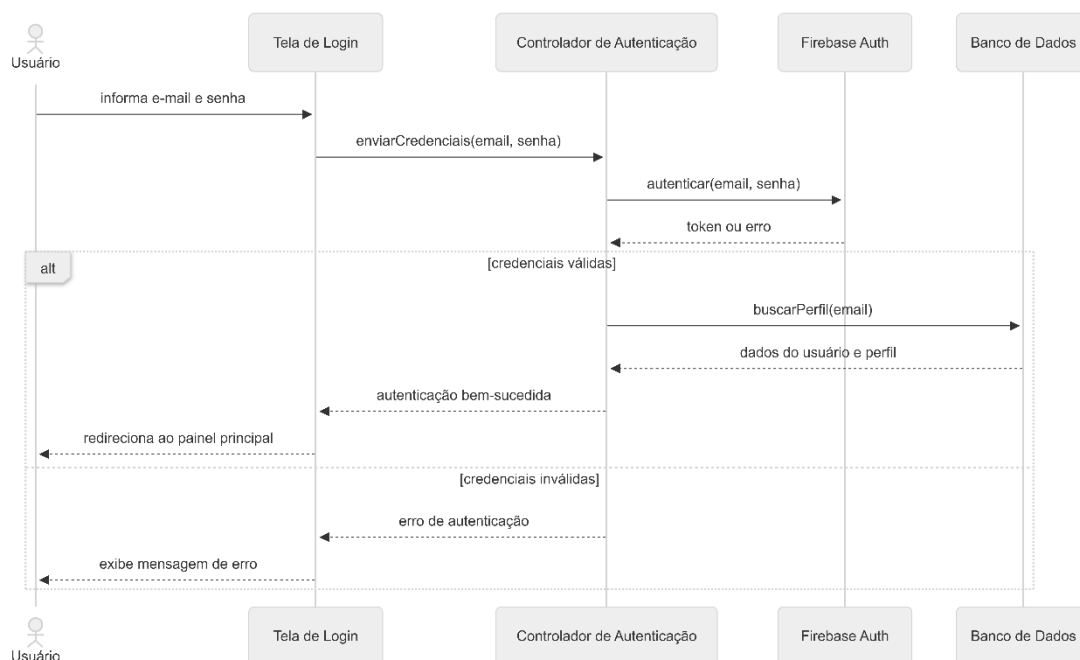


Fonte: Os autores.

4.4.3 Diagrama de Sequência

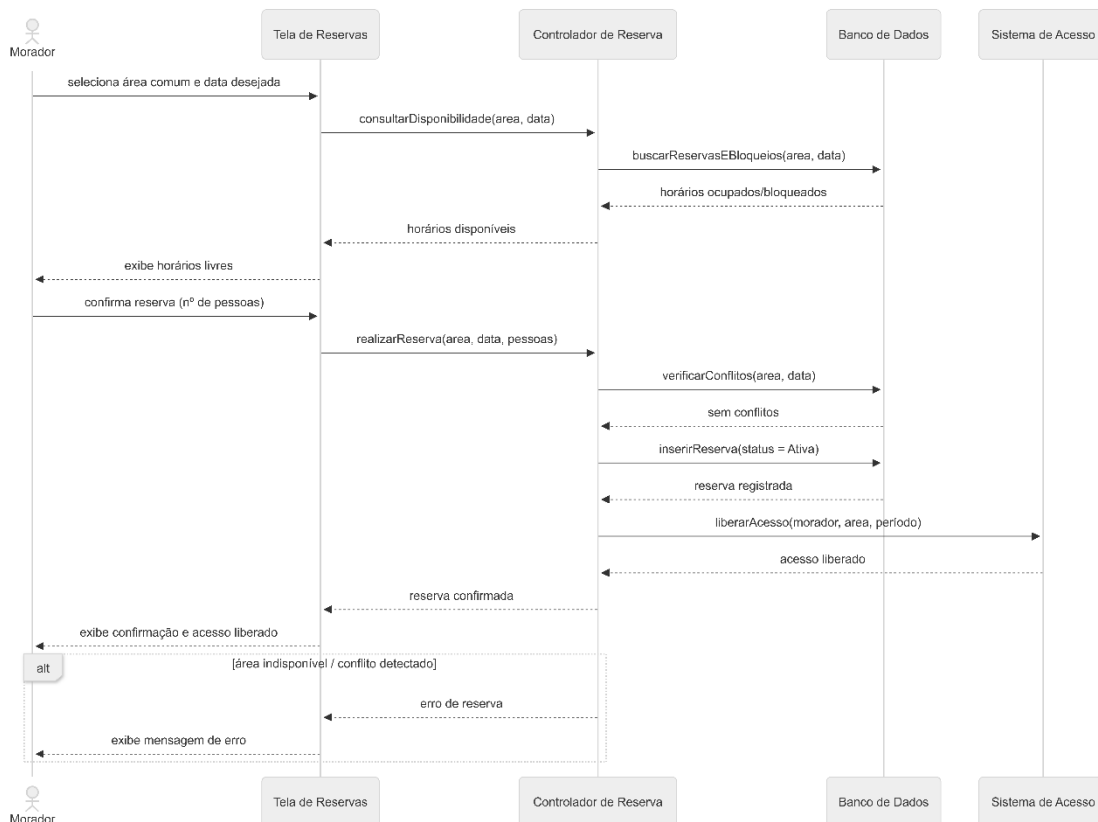
Os Diagramas de Sequência, apresentados nas Figuras 3 a 6, ilustram a interação entre os objetos do sistema ao longo do tempo para os fluxos principais. São apresentados os diagramas dos cenários de maior relevância para o entendimento do comportamento dinâmico do sistema OASIS. As mensagens trocadas seguem a mesma organização do padrão Boundary-Control-Entity adotado no Diagrama de Classes, percorrendo a sequência classe de fronteira (tela) → classe de controle (controlador) → classe de entidade (banco de dados) e, quando aplicável, os serviços externos Firebase e Intelbras.

Figura 3 – Diagrama de Sequência: Autenticar no Sistema (UC01)



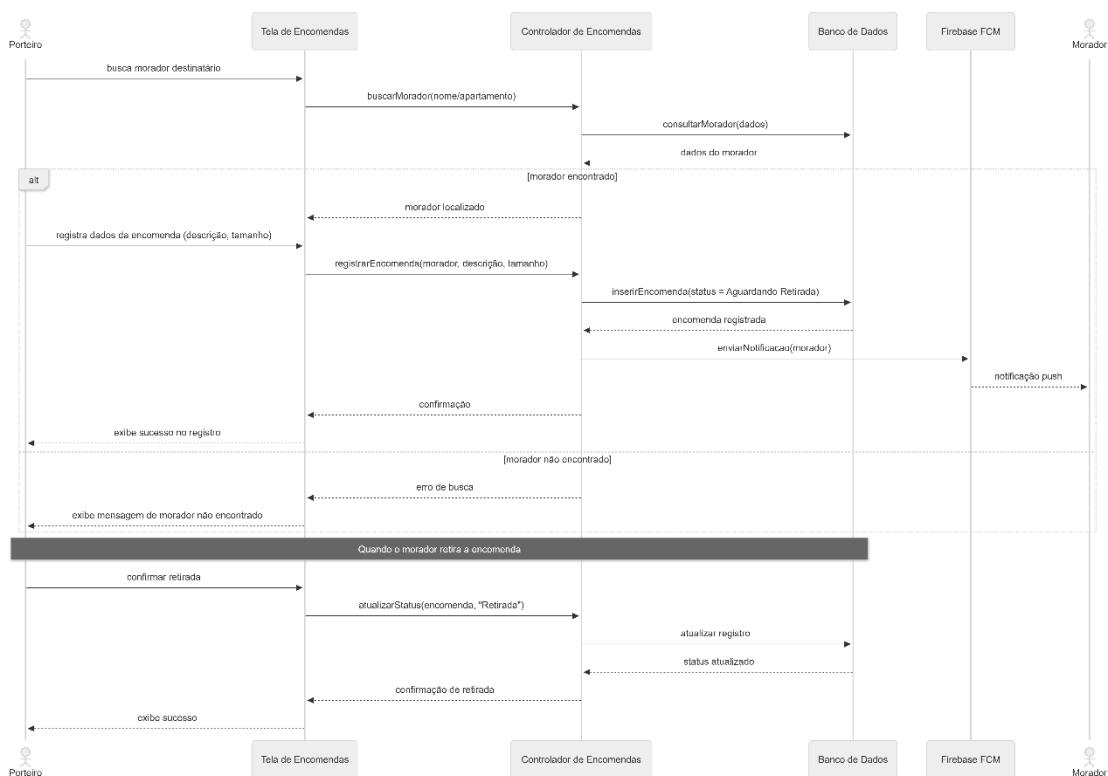
Fonte: Os autores.

Figura 4 – Diagrama de Sequência: Realizar Reserva (UC03)



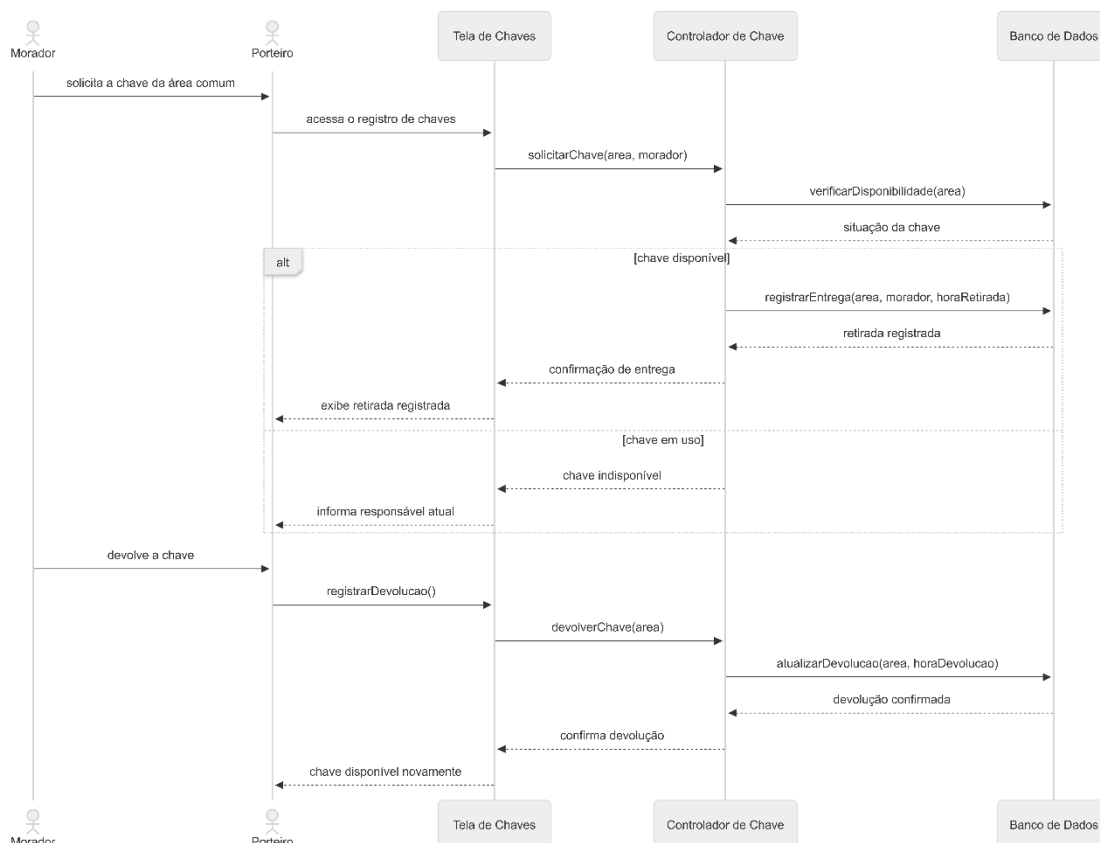
Fonte: Os autores.

Figura 5 – Diagrama de Sequência: Registrar Encomenda (UC07)



Fonte: Os autores.

Figura 6 – Diagrama de Sequência: Solicitar / Devolver Chave (UC06)



Fonte: Os autores.

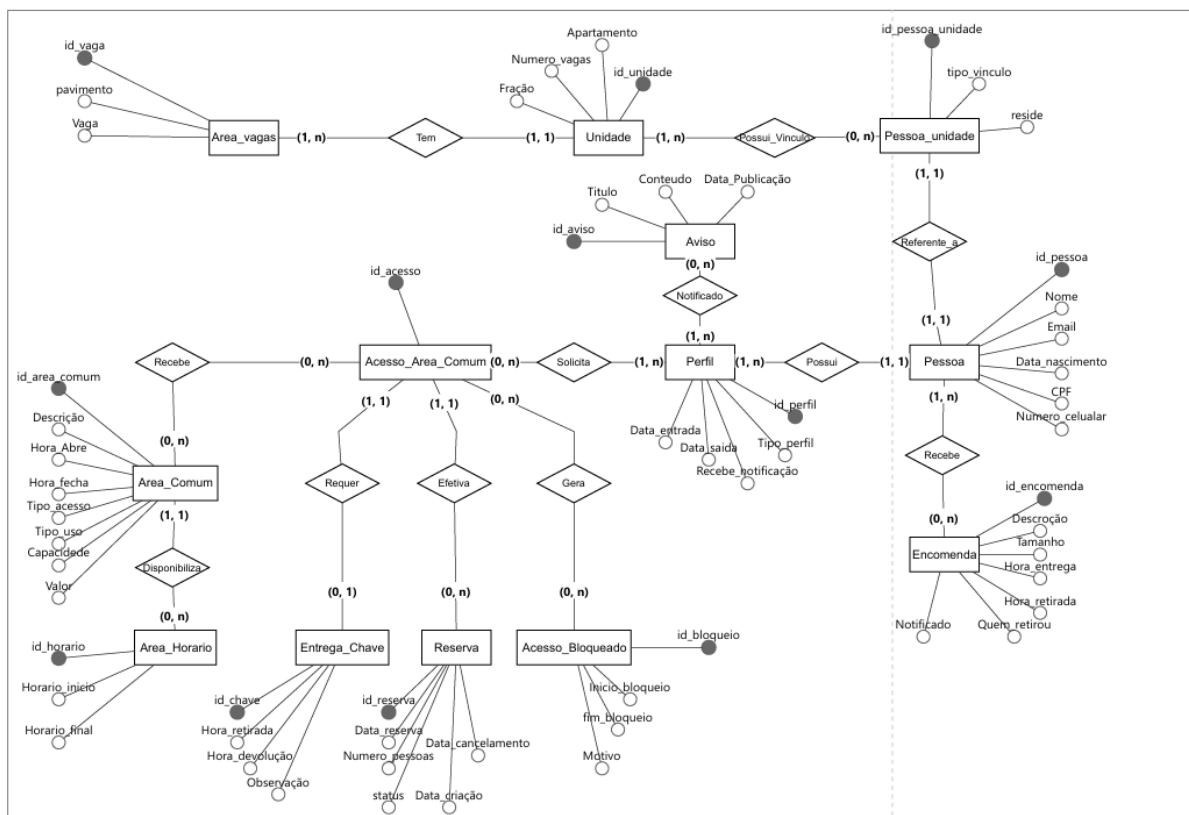
4.5 MODELAGEM DO BANCO DE DADOS

Esta seção apresenta a modelagem do banco de dados relacional do sistema OASIS, incluindo o Diagrama Entidade-Relacionamento (DER) conceitual, o Diagrama Lógico (Pé de Galinha), o Dicionário de Dados com a descrição completa de cada atributo e os scripts de criação do banco de dados em PostgreSQL.

4.5.1 Diagrama Entidade-Relacionamento

O Diagrama Entidade-Relacionamento (DER), apresentado na Figura 7, representa o modelo conceitual do banco de dados do OASIS. No modelo conceitual, não são representadas chaves estrangeiras, pois estas surgem naturalmente no mapeamento para o modelo lógico relacional. O DER foi elaborado com o software BRModelo.

Figura 7 – Diagrama Entidade-Relacionamento do Software OASIS



Fonte: Os autores.

4.5.2 Diagrama Lógico

O Diagrama Lógico (Pé de Galinha), apresentado na Figura 8, representa o mapeamento do modelo conceitual para o modelo relacional, com a identificação de todas as chaves primárias, chaves estrangeiras e restrições de integridade. O diagrama foi desenvolvido no BRModelo Web.

Figura 8 – Diagrama Lógico do Software OASIS



Fonte: Os autores.

4.5.3 Dicionário de Dados

O Dicionário de Dados descreve todas as tabelas e atributos do banco de dados do OASIS, apresentando para cada campo o tipo de dado, o tamanho, a restrição de nulidade, o tipo de chave, o domínio (conjunto de valores válidos), a descrição semântica e observações pertinentes. As Tabelas 4 a 16 apresentam o dicionário de cada entidade do sistema.

A Tabela 4 apresenta o dicionário de dados da entidade PESSOA. Armazena os dados das pessoas cadastradas no sistema (moradores, síndicos e porteiros).

Tabela 4 – Dicionário de Dados: PESSOA

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_pessoa	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único da pessoa	Gerado automaticamente
Nome	VARCHAR	100	NÃO	-	Texto	Nome completo da pessoa	-
EMAIL	VARCHAR	150	NÃO	-	E-mail válido, único	Endereço de e-mail para autenticação	Restrição UNIQUE

Data_nascimento	DATE	-	NÃO	-	Data (AAAA-MM-DD)	Data de nascimento da pessoa	-
cpf	INT	-	NÃO	-	Numérico, único	CPF da pessoa	Restrição UNIQUE
Numero_celular	INT	-	SIM	-	Numérico	Telefone celular de contato	Opcional

Fonte: Os autores.

A Tabela 5 apresenta o dicionário de dados da entidade PERFIL. Define o papel de cada pessoa no condomínio (morador, síndico ou porteiro) e suas preferências.

Tabela 5 – Dicionário de Dados: PERFIL

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_perfil	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único do perfil	Gerado automaticamente
id_pessoa	INT	-	NÃO	FK	Inteiro > 0	Pessoa associada ao perfil	FK → PESSOA(id_pessoa)
Data_entrada	DATE	-	NÃO	-	Data (AAAA-MM-DD)	Data de início do perfil	-
Data_saida	DATE	-	SIM	-	Data (AAAA-MM-DD)	Data de término do perfil	Nulo se perfil ativo
Recebe_notificacao	BOOLEAN	-	NÃO	-	{true, false}	Indica se o perfil recebe notificações	Padrão: true
Tipo_perfil	VARCHAR	20	NÃO	-	{Morador, Sindico, Porteiro}	Papel da pessoa no condomínio	-

Fonte: Os autores.

A Tabela 6 apresenta o dicionário de dados da entidade UNIDADE. Armazena as unidades (apartamentos) do condomínio.

Tabela 6 – Dicionário de Dados: UNIDADE

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_unidade	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único da unidade	Gerado automaticamente
Apartamento	INT	-	NÃO	-	Inteiro > 0	Número do apartamento	-
Numero_vagas	INT	-	NÃO	-	Inteiro >= 0	Quantidade de vagas da unidade	-
fracao	INT	-	SIM	-	Inteiro >= 0	Fração ideal da unidade	Opcional

Fonte: Os autores.

A Tabela 7 apresenta o dicionário de dados da entidade PESSOA_UNIDADE. Associa pessoas às unidades, representando o vínculo (proprietário, inquilino ou dependente).

Tabela 7 – Dicionário de Dados: PESSOA_UNIDADE

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_pessoa_unidade	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único do vínculo	Gerado automaticamente
id_pessoa	INT	-	NÃO	FK	Inteiro > 0	Pessoa vinculada à unidade	FK → PESSOA(id_pessoa)
id_unidade	INT	-	NÃO	FK	Inteiro > 0	Unidade vinculada à pessoa	FK → UNIDADE(id_unidade)
tipo_vinculo	VARCHAR	20	NÃO	-	{Proprietario, Inquilino, Dependente}	Natureza do vínculo da pessoa	-
reside	BOOLEAN	-	NÃO	-	{true, false}	Indica se a pessoa reside na unidade	Padrão: true

Fonte: Os autores.

A Tabela 8 apresenta o dicionário de dados da entidade AREA_VAGAS. Armazena as vagas de garagem vinculadas às unidades.

Tabela 8 – Dicionário de Dados: AREA_VAGAS

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_vaga	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único da vaga	Gerado automaticamente
idUnidade	INT	-	NÃO	FK	Inteiro > 0	Unidade proprietária da vaga	FK → UNIDADE(id_unidade)
Pavimento	VARCHAR	20	NÃO	-	{Terreo, Subsolo1, Subsolo2}	Pavimento onde fica a vaga	-
Vaga	VARCHAR	20	NÃO	-	{Simplex, Dupla, Coberta}	Tipo da vaga de garagem	-

Fonte: Os autores.

A Tabela 9 apresenta o dicionário de dados da entidade AREA_COMUM. Armazena as áreas comuns do condomínio passíveis de uso e reserva.

Tabela 9 – Dicionário de Dados: AREA_COMUM

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_area_comum	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único da área comum	Gerado automaticamente
Descricao	VARCHAR	100	NÃO	-	Texto	Nome ou descrição da área comum	-
Hora_Abre	DATETIME	-	NÃO	-	Hora	Horário de abertura da área	-
Hora_Fecha	DATETIME	-	NÃO	-	Hora	Horário de fechamento da área	Deve ser > Hora_Abre

Tipo_acesso	VARCHAR	20	NÃO	-	{Livre, Biometria, Chave}	Forma de controle de acesso à área	-
Tipo_uso	VARCHAR	20	NÃO	-	{Comum, Exclusivo, Reservavel}	Modo de utilização da área	-
Capacidade	INT	-	NÃO	-	Inteiro > 0	Capacidade máxima de pessoas	CHECK > 0
Observacoes	VARCHAR	255	SIM	-	Texto	Observações adicionais	Opcional
Valor	FLOAT	-	SIM	-	Decimal >= 0	Valor cobrado pelo uso (se houver)	Opcional

Fonte: Os autores.

A Tabela 10 apresenta o dicionário de dados da entidade AREA_HORARIO. Define as janelas de horário de funcionamento das áreas comuns.

Tabela 10 – Dicionário de Dados: AREA_HORARIO

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_horario	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único do horário	Gerado automaticamente
idArea_Comum	INT	-	NÃO	FK	Inteiro > 0	Área comum à qual o horário pertence	FK → AREA_COMUM(id_area_comum)
horario_inicio	DATETIME	-	NÃO	-	Hora	Horário de início de funcionamento	-
horario_final	DATETIME	-	NÃO	-	Hora	Horário de término de funcionamento	Deve ser > horario_inicio

Fonte: Os autores.

A Tabela 11 apresenta o dicionário de dados da entidade ACESSO_AREA_COMUM. Registra o acesso e uso efetivo de uma área comum por um perfil, base para reservas, chaves e bloqueios.

Tabela 11 – Dicionário de Dados: ACESSO_AREA_COMUM

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_acesso	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único do acesso	Gerado automaticamente
id_perfil	INT	-	NÃO	FK	Inteiro > 0	Perfil que efetuou o acesso	FK → PERFIL(id_perfil)
id_area_comum	INT	-	NÃO	FK	Inteiro > 0	Área comum acessada	FK → AREA_COMUM(id_area_comum)

Fonte: Os autores.

A Tabela 12 apresenta o dicionário de dados da entidade RESERVA. Armazena as reservas de áreas comuns realizadas a partir de um acesso.

Tabela 12 – Dicionário de Dados: RESERVA

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_reserva	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único da reserva	Gerado automaticamente
id_acesso	INT	-	NÃO	FK	Inteiro > 0	Acesso que originou a reserva	FK → ACESSO_AREA_COMUM(id_acesso)
Data_reserva	DATE	-	NÃO	-	Data (AAAA-MM-DD)	Data da reserva	-
Numero_Pessoas	INT	-	NÃO	-	Inteiro > 0	Quantidade de pessoas na reserva	-
status	VARCHAR	20	NÃO	-	{Ativa, Cancelada, Concluída}	Situação atual da reserva	Padrão: Ativa
Data_criacao	DATE	-	NÃO	-	Data (AAAA-MM-DD)	Data de criação do registro	Padrão: data atual
Data_cancelamento	DATE	-	SIM	-	Data (AAAA-MM-DD)	Data de cancelamento da reserva	Preenchido por gatilho

Fonte: Os autores.

A Tabela 13 apresenta o dicionário de dados da entidade ENTREGA_CHAVE. Registra a retirada e a devolução das chaves de áreas comuns na portaria.

Tabela 13 – Dicionário de Dados: ENTREGA_CHAVE

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_chave	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único da entrega de chave	Gerado automaticamente
idAcesso_area_comum	INT	-	NÃO	FK	Inteiro > 0	Acesso associado à entrega	FK → ACESSO_AREA_COMUM(id_acesso)
Hora_retirada	DATETIME	-	NÃO	-	Data e hora	Data e hora de retirada da chave	Padrão: data atual
Hora_devolucao	DATETIME	-	SIM	-	Data e hora	Data e hora de devolução da chave	Nulo enquanto não devolvida
observacao	VARCHAR	255	SIM	-	Texto	Observações sobre a entrega	Opcional

Fonte: Os autores.

A Tabela 14 apresenta o dicionário de dados da entidade ACESSO_BLOQUEADO. Registra os períodos de bloqueio de acesso a uma área comum para manutenção ou limpeza.

Tabela 14 – Dicionário de Dados: ACESSO_BLOQUEADO

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_bloqueio	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único do bloqueio	Gerado automaticamente
id_acesso	INT	-	NÃO	FK	Inteiro > 0	Acesso/área a que se refere o bloqueio	FK → ACESSO_AREA_COMUM(id_acesso)
inicio_bloqueio	DATE	-	NÃO	-	Data (AAAA-MM-DD)	Data de início do bloqueio	-
fim_bloqueio	DATE	-	NÃO	-	Data (AAAA-MM-DD)	Data de término do bloqueio	Deve ser > início
Motivo	VARCHAR	20	SIM	-	{Manutencao, Limpeza, Evento}	Motivo do bloqueio	Opcional

Fonte: Os autores.

A Tabela 15 apresenta o dicionário de dados da entidade ENCOMENDA. Registra o recebimento, a notificação e a retirada de encomendas na portaria, atuando também como aviso ao morador.

Tabela 15 – Dicionário de Dados: ENCOMENDA

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_encomenda	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único da encomenda	Gerado automaticamente
Descricao	VARCHAR	255	SIM	-	Texto	Descrição da encomenda	Opcional
Tamanho	VARCHAR	20	NÃO	-	{Pequeno, Medio, Grande}	Porte da encomenda	-
Hora_entrega	DATETIME	-	NÃO	-	Data e hora	Data e hora de recebimento na portaria	Padrão: data atual
Hora_retirada	DATETIME	-	SIM	-	Data e hora	Data e hora de retirada pelo morador	Nulo até a retirada
Quem_retirou	VARCHAR	20	SIM	-	{Proprio, Terceiro, Porteiro}	Quem retirou a encomenda	Nulo até a retirada
Notificado	BOOLEAN	-	NÃO	-	{true, false}	Indica se o morador foi notificado	Padrão: false
idPessoa	INT	-	NÃO	FK	Inteiro > 0	Pessoa destinatária da encomenda	FK → PESSOA(id_pessoa)

Fonte: Os autores.

A Tabela 16 apresenta o dicionário de dados da entidade AVISO. Armazena os avisos e notificações direcionados aos perfis do condomínio.

Tabela 16 – Dicionário de Dados: AVISO

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_aviso	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único do aviso	Gerado automaticamente
idPerfil	INT	-	NÃO	FK	Inteiro > 0	Perfil destinatário do aviso	FK → PERFIL(id_perfil)
Título	VARCHAR	200	NÃO	-	Texto	Título do aviso	-
Conteúdo	VARCHAR	500	NÃO	-	Texto	Conteúdo do aviso	-
Data_publicacao	DATETIME	-	NÃO	-	Data e hora	Data e hora de publicação	Padrão: data atual

Fonte: Os autores.

4.5.4 Scripts do Banco de Dados

Os scripts de criação do banco de dados do OASIS foram desenvolvidos para o PostgreSQL 16, seguindo as definições do Dicionário de Dados apresentado na seção 4.5.3. Todas as regras de negócio identificadas estão implementadas diretamente no banco por meio de restrições (CONSTRAINT), valores padrão (DEFAULT) e verificações (CHECK), conforme apresentado no Quadro 14.

Quadro 14 – Scripts de Criação do Banco de Dados (PostgreSQL 16)

```
-- =====
-- OASIS - Scripts de Criação do Banco de Dados
-- PostgreSQL 16
-- =====

CREATE TABLE pessoa (
    id_pessoa SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    data_nascimento DATE NOT NULL,
    cpf VARCHAR(14) NOT NULL UNIQUE,
    numero_celular VARCHAR(20)
);

CREATE TABLE unidade (
    id_unidade SERIAL PRIMARY KEY,
    apartamento INT NOT NULL,
    numero_vagas INT NOT NULL DEFAULT 0 CHECK (numero_vagas >= 0),
    fracao INT
);

CREATE TABLE pessoa_unidade (
    id_pessoa_unidade SERIAL PRIMARY KEY,
    id_pessoa INT NOT NULL,
    id_unidade INT NOT NULL,
```

```

    tipo_vinculo      VARCHAR(20)    NOT NULL,
    reside            BOOLEAN        NOT NULL DEFAULT TRUE,
    CONSTRAINT fk_pu_pessoa
        FOREIGN KEY (id_pessoa) REFERENCES pessoa(id_pessoa),
    CONSTRAINT fk_pu_unidade
        FOREIGN KEY (id_unidade) REFERENCES unidade(id_unidade),
    CONSTRAINT chk_pu_vinculo
        CHECK (tipo_vinculo IN ('Proprietario', 'Inquilino', 'Dependente'))
);

CREATE TABLE area_vagas (
    id_vaga          SERIAL PRIMARY KEY,
    id_unidade       INT           NOT NULL,
    pavimento        VARCHAR(20)   NOT NULL,
    vaga             VARCHAR(20)   NOT NULL,
    CONSTRAINT fk_vaga_unidade
        FOREIGN KEY (id_unidade) REFERENCES unidade(id_unidade),
    CONSTRAINT chk_vaga_pavimento
        CHECK (pavimento IN ('Terreo', 'Subsolo1', 'Subsolo2')),
    CONSTRAINT chk_vaga_tipo
        CHECK (vaga IN ('Simples', 'Dupla', 'Coberta'))
);

CREATE TABLE perfil (
    id_perfil        SERIAL PRIMARY KEY,
    id_pessoa        INT           NOT NULL,
    data_entrada     DATE          NOT NULL,
    data_saida       DATE,
    recebe_notificacao BOOLEAN     NOT NULL DEFAULT TRUE,
    tipo_perfil      VARCHAR(20)   NOT NULL,
    CONSTRAINT fk_perfil_pessoa
        FOREIGN KEY (id_pessoa) REFERENCES pessoa(id_pessoa),
    CONSTRAINT chk_perfil_tipo
        CHECK (tipo_perfil IN ('Morador', 'Sindico', 'Porteiro'))
);

CREATE TABLE area_comum (
    id_area_comum    SERIAL PRIMARY KEY,
    descricao        VARCHAR(100)  NOT NULL,
    hora_abre        TIME          NOT NULL,
    hora_fecha       TIME          NOT NULL,
    tipo_acesso      VARCHAR(20)   NOT NULL,
    tipo_uso         VARCHAR(20)   NOT NULL,
    capacidade       INT           NOT NULL CHECK (capacidade > 0),
    observacoes      VARCHAR(255),
    valor            FLOAT         CHECK (valor >= 0),
    CONSTRAINT chk_area_horario CHECK (hora_fecha > hora_abre),
    CONSTRAINT chk_area_acesso CHECK (tipo_acesso IN ('Livre', 'Biometria', 'Chave')),
    CONSTRAINT chk_area_uso    CHECK (tipo_uso IN ('Comum', 'Exclusivo', 'Reservavel'))
);

CREATE TABLE area_horario (
    id_horario       SERIAL PRIMARY KEY,
    id_area_comum    INT           NOT NULL,
    horario_inicio   TIME          NOT NULL,
    horario_final    TIME          NOT NULL,
    CONSTRAINT fk_horario_area
        FOREIGN KEY (id_area_comum) REFERENCES area_comum(id_area_comum)
        ON DELETE CASCADE,
    CONSTRAINT chk_horario_periodo
        CHECK (horario_final > horario_inicio)
);

CREATE TABLE acesso_area_comum (
    id_acesso        SERIAL PRIMARY KEY,
    id_perfil        INT           NOT NULL,
    id_area_comum    INT           NOT NULL,
    CONSTRAINT fk_acesso_perfil
        FOREIGN KEY (id_perfil) REFERENCES perfil(id_perfil),
    CONSTRAINT fk_acesso_area
        FOREIGN KEY (id_area_comum) REFERENCES area_comum(id_area_comum)
);

```

```

CREATE TABLE reserva (
    id_reserva SERIAL PRIMARY KEY,
    id_acesso INT NOT NULL,
    data_reserva DATE NOT NULL,
    numero_pessoas INT NOT NULL CHECK (numero_pessoas > 0),
    status VARCHAR(20) NOT NULL DEFAULT 'Ativa',
    data_criacao DATE NOT NULL DEFAULT CURRENT_DATE,
    data_cancelamento DATE,
    CONSTRAINT fk_reserva_acesso
        FOREIGN KEY (id_acesso) REFERENCES acesso_area_comum(id_acesso),
    CONSTRAINT chk_reserva_status
        CHECK (status IN ('Ativa', 'Cancelada', 'Concluida'))
);

CREATE TABLE entrega_chave (
    id_chave SERIAL PRIMARY KEY,
    id_acesso INT NOT NULL,
    hora_retirada TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    hora_devolucao TIMESTAMP,
    observacao VARCHAR(255),
    CONSTRAINT fk_chave_acesso
        FOREIGN KEY (id_acesso) REFERENCES acesso_area_comum(id_acesso),
    CONSTRAINT chk_chave_devolucao
        CHECK (hora_devolucao IS NULL OR hora_devolucao >= hora_retirada)
);

CREATE TABLE acesso_bloqueado (
    id_bloqueio SERIAL PRIMARY KEY,
    id_acesso INT NOT NULL,
    incio_bloqueio DATE NOT NULL,
    fim_bloqueio DATE NOT NULL,
    motivo VARCHAR(20),
    CONSTRAINT fk_bloqueio_acesso
        FOREIGN KEY (id_acesso) REFERENCES acesso_area_comum(id_acesso),
    CONSTRAINT chk_bloqueio_periodo
        CHECK (fim_bloqueio > incio_bloqueio),
    CONSTRAINT chk_bloqueio_motivo
        CHECK (motivo IN ('Manutencao', 'Limpeza', 'Evento'))
);

CREATE TABLE encomenda (
    id_encomenda SERIAL PRIMARY KEY,
    id_pessoa INT NOT NULL,
    descricao VARCHAR(255),
    tamanho VARCHAR(20) NOT NULL,
    hora_entrega TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    hora_retirada TIMESTAMP,
    quem_retirou VARCHAR(20),
    notificado BOOLEAN NOT NULL DEFAULT FALSE,
    CONSTRAINT fk_encomenda_pessoa
        FOREIGN KEY (id_pessoa) REFERENCES pessoa(id_pessoa),
    CONSTRAINT chk_encomenda_tamanho
        CHECK (tamanho IN ('Pequeno', 'Medio', 'Grande')),
    CONSTRAINT chk_encomenda_retirou
        CHECK (quem_retirou IS NULL OR quem_retirou IN ('Proprio', 'Terceiro',
'Porteiro'))
);

CREATE TABLE aviso (
    id_aviso SERIAL PRIMARY KEY,
    id_perfil INT NOT NULL,
    titulo VARCHAR(200) NOT NULL,
    conteudo VARCHAR(500) NOT NULL,
    data_publicacao TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_aviso_perfil
        FOREIGN KEY (id_perfil) REFERENCES perfil(id_perfil)
);

```

Fonte: Os autores. PostgreSQL 16.

4.5.5 Regras de Negócio e Gatilhos

Além das restrições declarativas (CONSTRAINT, CHECK e DEFAULT) apresentadas no Quadro 14, o sistema OASIS implementa regras de negócio dinâmicas por meio de gatilhos (triggers) no PostgreSQL. Os gatilhos garantem a consistência dos dados e automatizam ações decorrentes de eventos no banco, sem depender exclusivamente da camada de aplicação. O Quadro 15 descreve as regras de negócio implementadas e seus respectivos gatilhos, e o Quadro 16 apresenta sua codificação em PL/pgSQL.

Quadro 15 – Regras de Negócio implementadas por gatilhos

ID	Regra de Negócio	Descrição	Evento (Gatilho)
RN01	Registrar data de cancelamento	Ao alterar o status de uma reserva para 'Cancelada', o sistema preenche automaticamente o campo data_cancelamento com a data corrente.	RESERVA (BEFORE UPDATE)
RN02	Impedir reserva sobreposta	Antes de inserir uma reserva, o sistema verifica se já existe outra reserva ativa para a mesma área comum na mesma data, impedindo a inserção em caso de conflito.	RESERVA (BEFORE INSERT)
RN03	Impedir reserva em área bloqueada	Antes de inserir uma reserva, o sistema verifica se há bloqueio de acesso programado para a área comum na data solicitada, impedindo a reserva caso exista.	RESERVA (BEFORE INSERT)
RN04	Validar capacidade da reserva	Antes de inserir uma reserva, o sistema verifica se o número de pessoas informado não excede a capacidade da área comum.	RESERVA (BEFORE INSERT)
RN05	Registrar devolução de chave	Ao preencher a hora de devolução de uma entrega de chave, o sistema valida que a devolução é posterior à retirada.	ENTREGA_CHAVE (BEFORE UPDATE)
RN06	Notificar recebimento de encomenda	Após registrar uma encomenda, o sistema gera automaticamente um aviso ao perfil da pessoa destinatária e marca a encomenda como notificada.	ENCOMENDA (AFTER INSERT)
RN07	Marcar retirada de encomenda	Ao preencher a hora de retirada de uma encomenda, o sistema valida o preenchimento do responsável pela retirada (quem_retirou).	ENCOMENDA (BEFORE UPDATE)

Fonte: Os autores.

Quadro 16 – Implementação dos Gatilhos (PL/pgSQL)

```
-- =====
-- OASIS - Gatilhos (Triggers) - PostgreSQL 16
-- =====

-- RN01: registra data_cancelamento ao cancelar reserva
CREATE OR REPLACE FUNCTION fn_reserva_cancelamento()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.status = 'Cancelada'
        AND OLD.status <> 'Cancelada' THEN
        NEW.data_cancelamento := CURRENT_DATE;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```

CREATE TRIGGER tg_reserva_cancelamento
  BEFORE UPDATE ON reserva
  FOR EACH ROW
  EXECUTE FUNCTION fn_reserva_cancelamento();

-- RN02, RN03 e RN04: valida reserva (conflito, bloqueio e capacidade)
CREATE OR REPLACE FUNCTION fn_valida_reserva()
  RETURNS TRIGGER AS $$
  DECLARE
    v_area    INT;
    v_capac   INT;
  BEGIN
    -- area comum da reserva (via acesso)
    SELECT a.id_area_comum, ac.capacidade
      INTO v_area, v_capac
      FROM acesso_area_comum a
      JOIN area_comum ac ON ac.id_area_comum = a.id_area_comum
      WHERE a.id_acesso = NEW.id_acesso;

    -- RN02: reserva sobreposta na mesma area e data
    IF EXISTS (
      SELECT 1
      FROM reserva r
      JOIN acesso_area_comum a ON a.id_acesso = r.id_acesso
      WHERE a.id_area_comum = v_area
      AND r.status = 'Ativa'
      AND r.data_reserva = NEW.data_reserva
    ) THEN
      RAISE EXCEPTION 'Area indisponivel: ja existe reserva nesta data.';
    END IF;

    -- RN03: area bloqueada na data
    IF EXISTS (
      SELECT 1
      FROM acesso_bloqueado b
      JOIN acesso_area_comum a ON a.id_acesso = b.id_acesso
      WHERE a.id_area_comum = v_area
      AND NEW.data_reserva BETWEEN b.inicio_bloqueio AND b.fim_bloqueio
    ) THEN
      RAISE EXCEPTION 'Area bloqueada no periodo solicitado.';
    END IF;

    -- RN04: capacidade
    IF NEW.numero_pessoas > v_capac THEN
      RAISE EXCEPTION 'Numero de pessoas excede a capacidade da area.';
    END IF;

    RETURN NEW;
  END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tg_valida_reserva
  BEFORE INSERT ON reserva
  FOR EACH ROW
  EXECUTE FUNCTION fn_valida_reserva();

-- RN05: valida devolucao de chave (devolucao >= retirada)
CREATE OR REPLACE FUNCTION fn_valida_devolucao_chave()
  RETURNS TRIGGER AS $$
  BEGIN
    IF NEW.hora_devolucao IS NOT NULL
      AND NEW.hora_devolucao < NEW.hora_retirada THEN
      RAISE EXCEPTION 'Devolucao nao pode ser anterior a retirada.';
    END IF;
    RETURN NEW;
  END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tg_valida_devolucao_chave
  BEFORE UPDATE ON entrega_chave
  FOR EACH ROW

```

```

EXECUTE FUNCTION fn_valida_devolucao_chave();

-- RN06: gera aviso ao perfil do morador ao registrar encomenda
CREATE OR REPLACE FUNCTION fn_notifica_encomenda()
RETURNS TRIGGER AS $$
DECLARE
    v_perfil INT;
BEGIN
    -- perfil de morador da pessoa destinataria
    SELECT p.id_perfil INTO v_perfil
    FROM perfil p
    WHERE p.id_pessoa = NEW.id_pessoa
    AND p.tipo_perfil = 'Morador'
    LIMIT 1;

    IF v_perfil IS NOT NULL THEN
        INSERT INTO aviso (id_perfil, titulo, conteudo)
        VALUES (v_perfil, 'Nova encomenda',
        'Voce tem uma encomenda na portaria.');

```

Fonte: Os autores. PostgreSQL 16 / PL/pgSQL.

4.6 PROTÓTIPO DE INTERFACE

O protótipo de interface do OASIS foi desenvolvido na ferramenta Figma, contemplando as principais telas do sistema para os três perfis de usuários: Morador, Administração e Porteiro. As Figuras 9 a 12 apresentam as telas prototipadas para as funcionalidades de maior interação com o usuário.

Figura 9 – Protótipo: Tela de Login

O protótipo da tela de login para o sistema OASIS apresenta um design limpo e moderno. No topo, há um ícone de aplicativo (uma letra 'O' dentro de um círculo) e o nome 'OASIS' em uma fonte sans-serif em negrito. Abaixo do nome, o subtítulo 'Sistema de Gestão Condominial' é exibido em uma fonte menor. O formulário de login é composto por dois campos de entrada: 'E-mail', com um ícone de envelope e o texto 'seu@email.com' preenchido, e 'Senha', com um ícone de cadeado e caracteres ocultos por pontos. À direita do campo de senha, há um link 'Esqueci minha senha'. Abaixo dos campos, um botão de login em azul escuro com o texto 'Entrar' e uma seta para a direita. Na base do formulário, uma linha de texto orienta o usuário: 'Use: morador@email.com, síndico@email.com ou porteiro@email.com'. O formulário inteiro está centralizado sobre um fundo cinza claro.

Fonte: Os autores.

Figura 10 – Protótipo: Tela de Reserva de Espaços (Morador)

OASIS Carlos Silva CS

Dashboard
Reservas
Avisos
Encomendas
Chaves
Perfil

Reserva de Áreas Comuns

Selecione uma área e escolha o horário desejado

Academia
Capacidade: 10 pessoas
Antecedência: 1 dia

Piscina
Capacidade: 30 pessoas
Antecedência: 2 dias

Salão de Festas
Capacidade: 50 pessoas
Antecedência: 7 dias

Churrasqueira
Capacidade: 15 pessoas
Antecedência: 3 dias

Elevador de Serviço
Capacidade: 4 pessoas
Antecedência: mesmo dia

OASIS Carlos Silva CS

Dashboard
Reservas
Avisos
Encomendas
Chaves
Perfil

Reserva de Áreas Comuns

Selecione uma área e escolha o horário desejado

Academia
Capacidade: 10 pessoas

Selecione a Data
05/06/2026

Número de Pessoas
1

Regras de Reserva

- Antecedência mínima: 1 dia
- Capacidade máxima: 10 pessoas
- Cancelamento até 24h antes
- Máximo 2 reservas por semana

Horários Disponíveis - 04/06/2026

07:00 - 08:00	08:00 - 09:00
09:00 - 10:00	10:00 - 11:00
11:00 - 12:00	14:00 - 15:00
15:00 - 16:00	16:00 - 17:00
17:00 - 18:00	18:00 - 19:00
19:00 - 20:00	20:00 - 21:00
21:00 - 22:00	

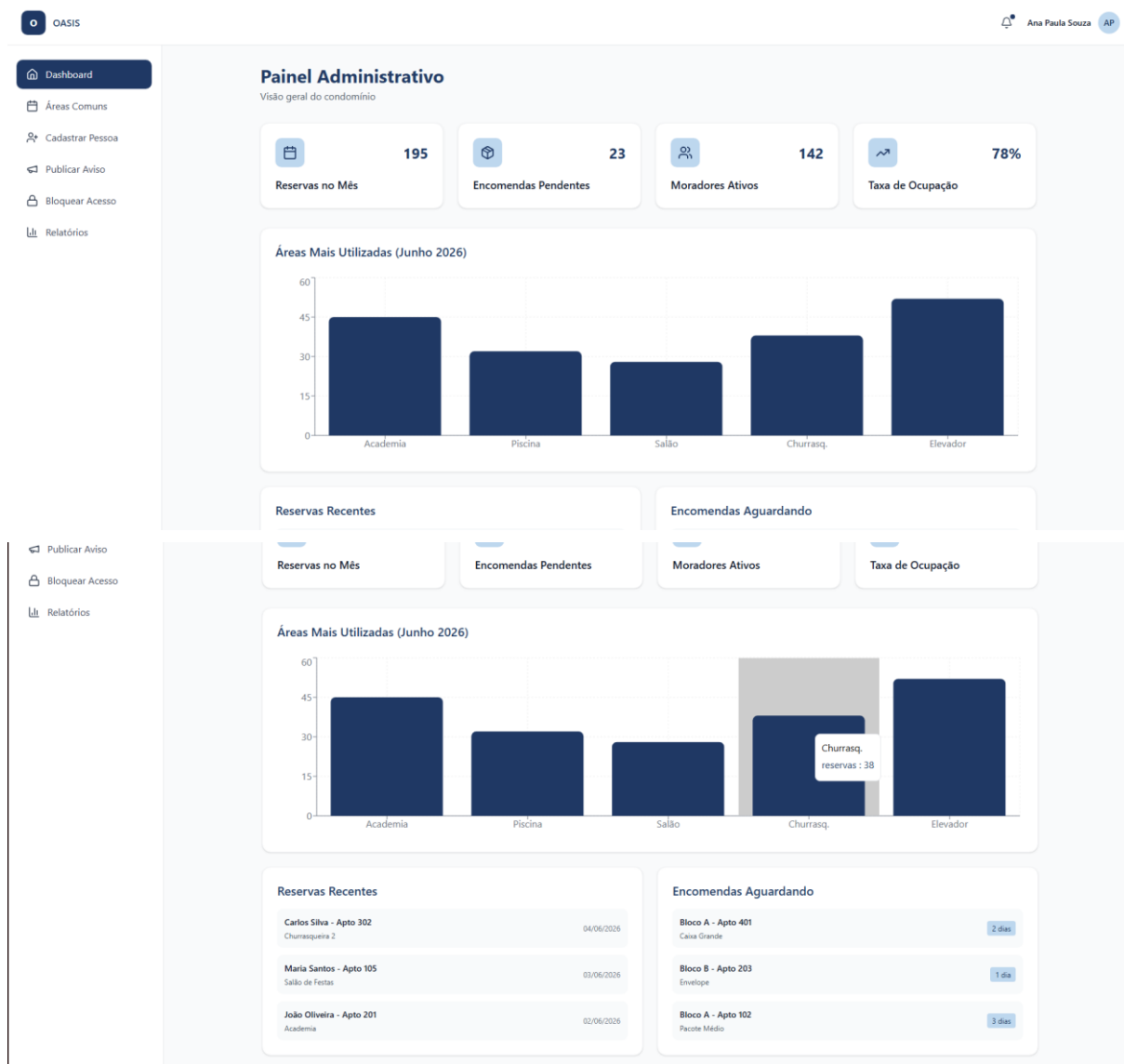
☒ Selecionado ☐ Ocupado ☐ Disponível

Confirmar Reserva

Voltar

Fonte: Os autores.

Figura 11 – Protótipo: Paineil Administrativo



Fonte: Os autores.

Figura 12 – Protótipo: Tela de Encomendas (Porteiro)

Gestão de Encomendas
Registre e controle as encomendas dos moradores

Registrar Encomenda

Morador
Selecione o morador

Descrição
Ex: Pacote Correios

Tamanho
Pequeno Médio Grande

Registrar Encomenda

Encomendas Registradas

Buscar por morador ou ap

Morador	Descrição	Tamanho	Status
Carlos Silva Bloco A - Apto 302	Pacote Correios - Médio		Aguardando ✓
Maria Santos Bloco B - Apto 105	Envelope - Pequeno		Aguardando ✓
João Oliveira Bloco A - Apto 201	Caixa Grande - Grande		Retirada
Ana Costa Bloco C - Apto 401	Documento - Pequeno		Aguardando ✓

Fonte: Os autores.

4.7 PLANO DE TESTES

4.7.1 Objetivo

Este Plano de Testes tem como objetivo definir a estratégia, o escopo, os recursos e os casos de teste necessários para verificar e validar o comportamento do sistema OASIS em conformidade com os requisitos funcionais estabelecidos nos Casos de Uso documentados na seção 4.4.1.

4.7.2 Escopo

Os testes abrangem todas as funcionalidades descritas nos casos de uso UC01 a UC13, cobrindo os fluxos principais e os principais fluxos alternativos identificados. Testes de desempenho e testes de segurança (penetração) estão fora do escopo deste plano.

4.7.3 Estratégia e Critérios de Aceitação

A estratégia adotada é a de testes funcionais caixa-preta, em que as entradas e saídas são verificadas sem conhecimento da implementação interna. Os critérios de aceitação são: (a) o resultado obtido deve corresponder exatamente ao resultado esperado descrito em cada caso de teste; (b) nenhum caso de teste de prioridade alta deve apresentar falha; (c) os casos de teste de prioridade média devem ter taxa de aprovação mínima de 90%.

4.7.4 Casos de Teste

O Quadro 17 apresenta os casos de teste elaborados para o sistema OASIS, com identificação, caso de uso relacionado, pré-condição, dados de entrada, resultado esperado e tipo de teste.

Quadro 17 – Casos de Teste do Sistema OASIS

ID	Caso de Teste	UC	Pré-condição	Entrada	Resultado Esperado	Tipo
CT01	Login com credenciais válidas	UC01	Usuário cadastrado	E-mail e senha válidos	Redireciona ao painel do perfil	Funcional
CT02	Login com senha inválida	UC01	Usuário cadastrado	E-mail válido, senha errada	Exibe mensagem de erro; mantém na tela de login	Funcional
CT03	Cadastro de morador com dados completos	UC11	Admin autenticado	Todos os campos obrigatórios preenchidos	Morador cadastrado; e-mail de credenciais enviado	Funcional
CT04	Cadastro com e-mail já existente	UC11	Admin autenticado	E-mail já cadastrado no sistema	Sistema exibe mensagem de e-mail duplicado	Funcional
CT05	Consulta de disponibilidade de área	UC02	Morador autenticado	Seleção de área e data	Calendário exibe horários livres e ocupados	Funcional
CT06	Consulta de área bloqueada	UC02	Morador autenticado; área com bloqueio ativo	Seleção de área no período bloqueado	Sistema exibe aviso de indisponibilidade com motivo	Funcional

CT07	Reserva de área disponível	UC03	Morador autenticado; horário disponível	Área, data e horário válidos	Reserva registrada; acesso liberado para o período	Funcional
CT08	Reserva com limite de unidade excedido	UC03	Morador com 2 reservas ativas (limite: 2)	Tentativa de nova reserva	Sistema bloqueia e exibe mensagem de limite atingido	Funcional
CT09	Reserva em horário sobreposto	UC03	Área com reserva ativa no horário	Mesmo horário de reserva existente	Gatilho impede a reserva e exibe erro de sobreposição	Funcional
CT10	Cancelamento dentro do prazo	UC04	Reserva ativa dentro do prazo	Confirmação de cancelamento	Reserva cancelada; data de cancelamento registrada; horário liberado	Funcional
CT11	Cancelamento fora do prazo	UC04	Reserva ativa fora do prazo de cancelamento	Tentativa de cancelamento	Sistema exibe mensagem de prazo expirado	Funcional
CT12	Visualização do mural de avisos	UC05	Morador autenticado; avisos cadastrados	Acesso ao mural	Avisos exibidos em ordem cronológica decrescente	Funcional
CT13	Empréstimo de chave disponível	UC06	Porteiro autenticado; chave disponível	Chave e morador solicitante	Empréstimo registrado; chave marcada como 'Emprestada'	Funcional
CT14	Empréstimo de chave já emprestada	UC06	Chave com status 'Emprestada'	Tentativa de novo empréstimo	Sistema exibe responsável atual e impede o empréstimo	Funcional
CT15	Devolução de chave	UC06	Empréstimo ativo sem devolução	Registro de devolução	Devolução registrada; chave marcada como 'Disponível'	Funcional
CT16	Registro de encomenda	UC07	Porteiro autenticado; morador cadastrado	Morador destinatário e descrição	Encomenda registrada; notificação gerada ao morador	Funcional
CT17	Registro com morador inexistente	UC07	Porteiro autenticado	Morador não cadastrado	Sistema exibe mensagem de morador não encontrado	Funcional
CT18	Confirmação de retirada de encomenda	UC08	Porteiro autenticado; encomenda aguardando	Confirmação de retirada	Status atualizado para 'Retirada'; data de retirada registrada	Funcional
CT19	Gerenciamento de área comum	UC09	Admin autenticado	Nome, capacidade, horários e utensílios	Área registrada e disponível para reservas	Funcional
CT20	Publicação de aviso no mural	UC10	Admin autenticado	Título e conteúdo preenchidos	Aviso publicado; notificações geradas aos moradores	Funcional
CT21	Bloqueio de acesso de morador	UC12	Admin autenticado; morador ativo	Seleção de morador e ação 'bloquear'	Permissão atualizada e sincronizada com o dispositivo	Integração
CT22	Emissão de relatório de uso	UC13	Admin autenticado; dados no período	Tipo de relatório e período	Relatório consolidado exibido e disponível para exportação	Funcional

Fonte: Os autores.

5 CRONOGRAMA

O cronograma apresenta as atividades que serão executadas pela equipe de desenvolvimento do OASIS ao longo do ano letivo de 2026, organizadas por mês. A Tabela 3 apresenta a sequência das atividades, com os meses de execução destacados em azul.

Tabela 3 – Cronograma de atividades do projeto OASIS

Atividade do Projeto	Mar	Abr	Mai	Jun	Jul	Ago	Set	Out	Nov	Dez
Levantamento de Requisitos										
Elaboração do Anteprojeto										
Modelagem Preliminar (Casos de Uso e Classes)										
Modelagem do Banco de Dados (DER e Lógico)										
Refinamento da Modelagem UML										
Prototipação de Interface (Figma)										
Definição de Arquitetura e Padrões										
Elaboração do Plano de Testes										
Configuração do Ambiente de Desenvolvimento										
Codificação do Backend (API REST)										
Codificação do Frontend Web										
Integração com API Intelbras										
Integração Frontend + Backend										
Implementação de Testes Automatizados										
Homologação e Correções										
Implantação do Sistema										
Treinamento da Cliente										
Elaboração do Manual do Usuário										
Documentação Final										

Fonte: Os autores.

6 CONCLUSÃO

A análise do cenário atual do condomínio evidencia que a solução vigente não atende plenamente às necessidades da administração e dos moradores, especialmente no que diz respeito à integração entre reservas de espaços coletivos e controle de acesso biométrico, ao controle de encomendas, à comunicação centralizada por meio de mural de avisos e ao monitoramento do fluxo de moradores. O projeto OASIS surge como uma resposta direta a essas demandas, propondo um sistema focado, integrado e eficiente.

Com base no levantamento de requisitos junto à cliente, na análise da situação atual e na modelagem preliminar realizada, conclui-se que o projeto é tecnicamente viável dentro das condições apresentadas. O conjunto de tecnologias selecionado para o desenvolvimento — React, Node.js, PostgreSQL, Firebase e integração via API Intelbras — é bem documentado, amplamente adotado no mercado e compatível com o conhecimento técnico da equipe desenvolvedora.

O escopo foi delimitado de forma realista, priorizando as funcionalidades de maior valor para a cliente e reservando a expansão do sistema para versões futuras. A definição da plataforma web exclusiva permite reduzir significativamente os custos do projeto sem prejudicar a experiência do usuário, dada a natureza responsiva da aplicação proposta. O cronograma é fiel ao calendário oficial da disciplina, contemplando todas as datas de entrega e as quatro bancas de avaliação previstas.

Portanto, o desenvolvimento do OASIS é considerado viável e pertinente, com potencial de impacto positivo direto na segurança, organização e qualidade de vida da comunidade condominial atendida.

7 REFERÊNCIAS

PRESSMAN, R. S.; MAXIM, B. R. Engenharia de software: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

SOMMERVILLE, I. Engenharia de software. 10. ed. São Paulo: Pearson, 2018.

FOWLER, M. UML distilled: a brief guide to the standard object modeling language. 3. ed. Boston: Addison-Wesley, 2004.

GLASSDOOR. *Salários de Desenvolvedor Pleno no Brasil*. 2026. Disponível em: https://www.glassdoor.com.br/Salarios/desenvolvedor-pleno-salario-SRCH_KO0,19.htm. Acesso em: mar. 2026.

REACT. *React – A JavaScript library for building user interfaces*. 2026. Disponível em: <https://react.dev>. Acesso em: mar. 2026.

NODE.JS. *Node.js Documentation*. 2026. Disponível em: <https://nodejs.org/en/docs>. Acesso em: mar. 2026.

NESTJS. *NestJS – A progressive Node.js framework*. 2026. Disponível em: <https://docs.nestjs.com>. Acesso em: mar. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL Documentation*. 2026. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: mar. 2026.

FIREBASE. *Firebase Documentation*. Google. 2026. Disponível em: <https://firebase.google.com/docs>. Acesso em: mar. 2026.

INTELBRAS. *Documentação da API de Controle de Acesso*. 2026. Disponível em: <https://www.intelbras.com/pt-br/>. Acesso em: mar. 2026.

MERMAID. *Mermaid – Diagramming and charting tool*. 2026. Disponível em: <https://mermaid.js.org>. Acesso em: maio 2026.

8 RESPONSABILIDADES



Documento assinado digitalmente
JOAO VITOR CARVALHO DE ANDRADE
Data: 02/06/2026 08:06:30-0300
Verifique em <https://validar.iti.gov.br>